

CANN-Bench开源全面解读

<https://gitcode.com/cann/cann-bench>

CANN

AI赋能CANN开发：新机遇与新挑战

随着大语言模型在代码生成领域的能力快速提升，各种CodingAgent和Skill生态快速发展，AI能完成复杂系统级开发任务，CANN领域目前也在大规模的应用AI进行赋能，但同时也遇到了很多挑战。

机
遇



模型能力持续提升

- **代码生成**：头部开/闭源模型接进专家水平
- **上下文窗口**：从200k到1M大幅提升
- **推理成本**：API价格持续下降



Agent Harness日趋完善

- **工具集成**：git、CI、仿真等工具链集成
- **反馈机制**：运行报错、精度偏差自动归因修复
- **自动化闭环**：从“代码生成”到“编译-运行-精度验证-性能采集”全链路自动化



昇腾生态日益成熟

- **硬件迭代**：Atlas系列持续升级，架构不断迭代
- **社区活跃**：开发者数量倍增，技术研讨活跃
- **行业落地**：支撑关键开源模型训推首发，行业份额持续提升

挑
战



昇腾算子Arena榜单尚属空白

- 目前尚多为内部测试、单点demo/showcase
- 没有公开、统一评测、持续维护的榜单平台来沉淀进展
- 学界、产业、社区需要在同一把尺子上对话



缺少Agent-Native Benchmark

- **Agent-Native**：高效公正的自动化评测体系
- **Anti-Hacking机制**：覆盖cpu计算卸载 / 标准篡改 / 结果缓存等多种Hacking机制，防止Agent-Hacking分数失真



缺少统一的多维评测标准

- **多维评测**：需综合精度、泛化性、性能等多个评测维度
- **算子场景众多**：语言/多模态/推荐模型；训练/推理
- **基线标准**：缺乏官方基线 + 性能理论上限

CANN-Bench简介

An Agent native benchmark for CANN

它是专为昇腾CANN平台打造的**自动化评测体系**。

通过**标准化**的流程、工具和平台，为AI模型和算子在CANN平台上的运行提供客观、稳定的评测基准。



核心定位：基准测试 (Benchmarking) 体系

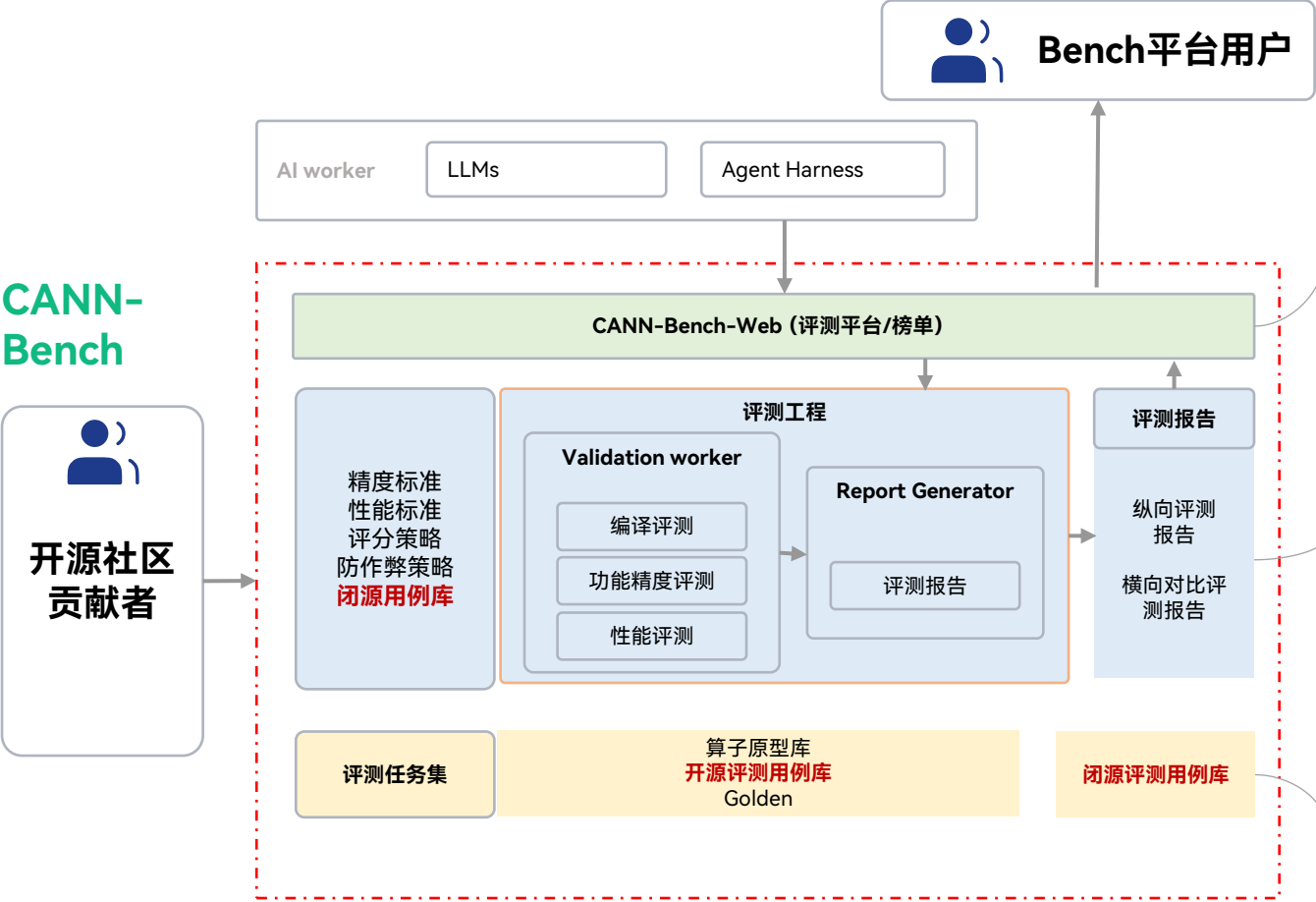
专注于建立专业的基准测试体系，包含可扩展的评测集、工业级的精度性能评测方法、自动化评测工程 and 平台，解决评测流程不一、结果难对齐的问题。



关键价值：Agent-Native · 客观公正 · 鲁棒性

- Agent-Native的高效自动化Benchmark
- 精度 + 泛化 + 性能等多维度评测
- 双性能标杆：官方性能基线 + 理论上限
- 建立防作弊机制，杜绝Agent Hacking刷分失真

CANN-Bench三层架构设计



平台层(Platform Layer)

核心职责

提供在线评测服务和榜单展示。

关键组件

在线评测平台、Leaderboard榜单系统



评测层(Evaluation Layer)

核心职责

自动高效的调度执行评测任务，是整个系统的核心引擎。

关键组件

精度评测、性能评测、自动化并行调度工程、安全防护。



数据层(Data Layer)

核心职责

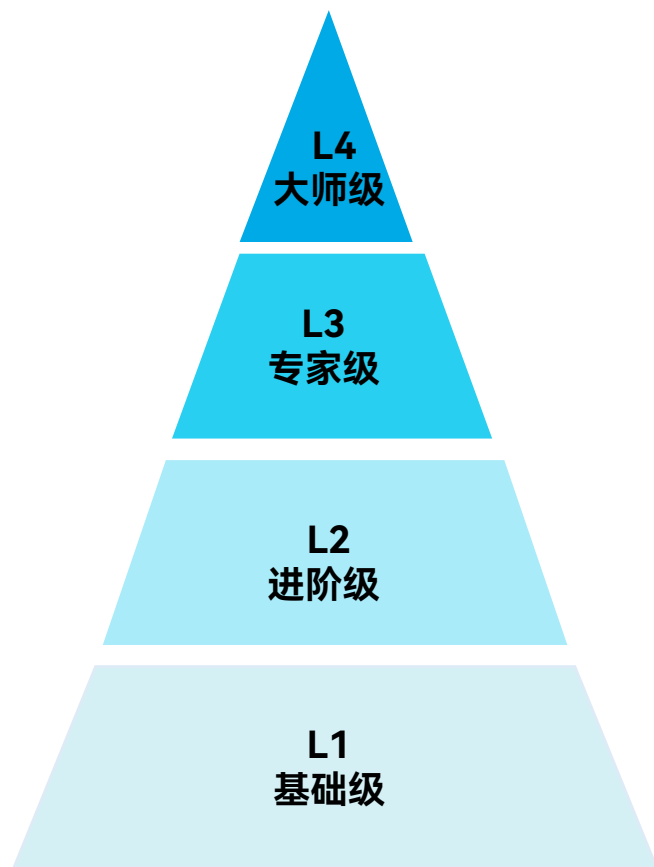
提供评测提供标准化、高质量的输入数据。

关键组件

标准化任务集、标准化工程样例、性能基线数据。

数据层-评测任务用例分级

根据计算流、计算模式，将评测算子分为四个难度等级，不同等级在总分计量中会有不同的**分值权重**，以更科学地衡量综合能力



代表算子:
FlashAttentionScore、
LSTM、MLA

代表算子:
MHCSinkhorn、Conv2D、
QuantMatmul、Engram

代表算子:
Gather、ArgMax、
ApplyAdamW、LayerNorm

代表算子:
Exp、Sigmoid、
MaskedScale、Gelu

L4 大师级

复杂CV融合算子、Attentions、Transformer-block等

L3 专家级

向量融合算子、张量算子(矩阵乘/卷积)、初阶CV融合算子等

L2 进阶级

进阶向量算子、MIMO(多输入输出)、归约类、索引类等

L1 基础级

简单向量算子、SISO(单输入单输出)、Element-wise、激活函数等

数据层 - 评测标准化输入

构建标准化的评测任务集合，是整个评测体系的基础，定义输入的标准规范，确保了评测的公平和可复现，同时标准也有利于构建可扩展的生态

算子定义 · proto.yaml

标准化描述算子接口、参数类型与约束条件

$$y = e^{(x \cdot scale + shift) \cdot \ln(base)}, base > 0$$

$$y = e^{x \cdot scale + shift}, base \leq 0$$

条件	简化公式
$base \leq 0, scale=1, shift=0$	$y=exy=ex$
$base > 0, scale=1, shift=0$	$y=basexy=basex$
$base=1$ (任意 $scale, shift$)	$y=1y=1$ (因 $\ln 1=0\ln 1=0$)

```
operator:
  name: Exp
  category: Elementwise
  difficulty: L1
  formula: y = e^((x * scale +
  shift) * ln(base))
  description: 计算输入张量的指数函数,
  支持自定义底数、缩放和偏移
  shape_support: 输入任意维度, 输出与
  输入相同shape
  attrs:
    - name: base
      type: float
      default: -1.0
  ...
```

算子用例 · cases.csv

包含多种边界与典型场景的测试输入数据集

	operator	case_id	input_shape	dtype	attrs	value_range	baseline_perf_us
1	Exp	1	[[1024, 1024]]	["float16"]	{"base": -1.0, "scale": 1.0, "shift": 0.0}	[-1, 1]	18.22
2	Exp	2	[[2048, 2048]]	["float32"]	{"base": -1.0, "scale": 1.5, "shift": 0.0}	[-2, 2]	38.82
3	Exp	3	[[4096, 4096]]	["bfloat16"]	{"base": -1.0, "scale": 2.0, "shift": 0.0}	[-3, 3]	68.74
4	Exp	4	[[8192, 8192]]	["float16"]	{"base": -1.0, "scale": 0.5, "shift": 0.0}	[-10, 10]	566.11
5	Exp	5	[[8192, 8192]]	["float32"]	{"base": -1.0, "scale": 1.0, "shift": 1.0}	[-100, 100]	1228.01
6	Exp	6	[[1023, 1023]]	["bfloat16"]	{"base": 2.0, "scale": 1.0, "shift": 0.0}	[-0.1, 0.1]	21.70
7	Exp	7	[[1009, 1021]]	["float16"]	{"base": 2.0, "scale": 1.5, "shift": 0.0}	[-1, 2]	23.34
8	Exp	8	[[1537, 769]]	["float32"]	{"base": 10.0, "scale": 1.0, "shift": 0.0}	[-5, 10]	27.24
9	Exp	9	[[363, 367, 373]]	["bfloat16"]	{"base": -1.0, "scale": 2.0, "shift": 0.5}	[-50, 100]	224.12

Golden 脚本 · golden.py

提供精确的计算真值，用于验证算子正确性

```
def exp(
    x: torch.Tensor, base: float = -1.0, scale: float = 1.0,
    shift: float = 0.0
) -> torch.Tensor:
    """
    计算输入张量的指数函数
    - base <= 0: y = exp(scale * x + shift)
    - base > 0: y = exp((shift + scale * x) * ln(base))
    """
    # 检测输入 dtype
    input_dtype = x.dtype
    # FP16/BF16 输入需要升到 FP32 计算以保证精度
    # FP32/FP64 输入保持原样计算
    if input_dtype in (torch.float16, torch.bfloat16):
        compute_dtype = torch.float32
    else:
        compute_dtype = input_dtype
    # 转换到计算精度
    x_compute = x.to(compute_dtype)
    temp = scale * x_compute + shift
    if base > 0:
        temp = temp * torch.log(torch.tensor(base,
        dtype=temp.dtype, device=temp.device))
    y = torch.exp(temp)
    # 转回原始 dtype
    if input_dtype in (torch.float16, torch.bfloat16):
        return y.to(input_dtype)
    return y
```

数据层 - 评测标准化输入

多种工程样例(Ascend C) 供测试用户选择; 未来将加入多种kernel编程语言(PYPTO/TileLang/Triton) + 配套工程样例支持

标准工程 • {}_launch_example

标准算子代码框架

```
acInn_launch_example/
├── cann_bench/           # Python包
├── cmake/               # 公共CMake配置(不感知算子)
│   ├── func.cmake      # 注册宏定义
│   └── ...
├── csrc/
│   ├── extension.cpp   # Python扩展入口
│   └── ops/            # 算子目录
│       ├── CMakeLists.txt # 自动发现算子
│       ├── add/        # Add算子
│       │   ├── CMakeLists.txt # 算子自注册
│       │   ├── op_host/    # 算子Host实现
│       │   ├── op_kernel/  # 算子Kernel实现
│       │   ├── op_api/    # 算子API实现
│       │   └── op_plugin/  # 算子插件实现
│       └── sqrt/         # Sqrt算子(结构相同)
├── dist/                # 输出目录
│   ├── cann_bench_YYYYMMDD_linux_aarch64.run # run包
│   └── cann_bench-1.0.0-*.whl                 # wheel包
├── scripts/            # 打包脚本
│   ├── build_run.sh
│   ├── build_wheel.sh
│   └── build.sh        # 统一构建入口
└── and setup.py
```

```
# 构建 wheel 包
bash build.sh
# 构建 + 安装
bash build.sh --install
# Python 使用
import cann_bench
z = cann_bench.add(x, y) # 或 torch.ops.cann_bench.add(x, y)
```

<https://gitcode.com/cann/cann-bench>

Aclnn工程 aclnn_launch

```
add/
├── CMakeLists.txt      # 算子自注册 (调用 register_aclnn_op)
├── op_host/
│   ├── add_def.cpp     # 算子定义 (输入/输出类型声明)
│   ├── add_infershape.cpp # Shape 推导
│   └── add_tiling.cpp   # Tiling 实现
├── op_kernel/
│   ├── add.h           # Kernel 头文件
│   ├── add_kernel.cpp  # Kernel 实现
│   ├── add_tiling_data.h # Tiling 数据结构
│   └── add_tiling_key.h # Tiling Key 定义
├── op_api/
│   ├── aclnn_add.cpp    # L2 API (aclnnMulGetWorkspaceSize, aclnnMul)
│   ├── aclnn_add.h
│   ├── add.cpp          # L0 API
│   └── add.h
└── op_plugin/
    └── add_plugin.cpp   # Python bindings (torch.ops.cann_bench.add)
```

算子直调工程 direct_launch

```
add/
├── CMakeLists.txt      # 算子自注册
├── op_kernel/
│   ├── add_kernel.cpp  # Kernel + Tiling + Launch (bisheng 编译)
│   └── add_launch.h    # Launch 函数声明 (g++ 可见)
└── op_plugin/
    └── add_plugin.cpp   # torch.library 注册
```

更多工程接入样例...

评测层-精度标准&对比方案

社区和业界在算子评测集中一般采用相对误差作为标准，存在一定的局限性，无法满足算子工业场景的使用。CANN-Bench基于工业应用场景，引入了更加科学的精度标准体系。

Golden: 3.140625

0100001001001000

FP16: 0x4248

Golden + 1 ulp: 3.142578 (Accepted)

0100001001001001

FP16: 0x4249

Golden + 4 ulp: 3.148438 (Rejected)

0100001001001100

FP16: 0x424C

精度阈值表		
数据类型	精度阈值 (Threshold)	MARE 阈值 (10×Threshold)
float16	$2^{-10} \approx 0.00098$	$2^{-6} \approx 0.0156$
bfloat16	$2^{-7} \approx 0.00781$	$2^{-3} \approx 0.125$
float32	$2^{-13} \approx 0.00012$	$2^{-9} \approx 0.002$

小值域阈值表		
数据类型	小值域阈值 (Small Value Threshold)	小值域误差阈值 (Small Value Error)
float16	$2^{-11} \approx 4.88 \times 10^{-4}$	$2^{-16} \approx 1.53 \times 10^{-5}$
bfloat16	$2^{-8} \approx 3.91 \times 10^{-3}$	$2^{-16} \approx 1.53 \times 10^{-5}$
float32	$2^{-14} \approx 6.10 \times 10^{-5}$	$2^{-30} \approx 9.31 \times 10^{-10}$

正常阈值误差标准

$$\text{MERE} = \text{avg} \left(\frac{|\text{actual} - \text{golden}|}{|\text{golden}| + \epsilon} \right)$$
$$\text{MARE} = \max \left(\frac{|\text{actual} - \text{golden}|}{|\text{golden}| + \epsilon} \right)$$
$$\text{MERE} < \text{threshold} \quad \wedge \quad \text{MARE} < 10 \times \text{threshold}$$

小值域误差标准

数据类型	小值域阈值 (Small Value Threshold)	小值域误差阈值 (Small Value Error)
float16	$2^{-11} \approx 4.88 \times 10^{-4}$	$2^{-16} \approx 1.53 \times 10^{-5}$
bfloat16	$2^{-8} \approx 3.91 \times 10^{-3}$	$2^{-16} \approx 1.53 \times 10^{-5}$
float32	$2^{-14} \approx 6.10 \times 10^{-5}$	$2^{-30} \approx 9.31 \times 10^{-10}$

$$\frac{\text{ErrorCount}_{\text{npu}}}{\max(\text{ErrorCount}_{\text{cpu}}, 1)} \leq 2$$

相消位置误差标准

$$\frac{\text{CancelErrorCount}_{\text{npu}}}{\max(\text{CancelErrorCount}_{\text{cpu}}, 1)} \leq 2$$

理论依据: IEEE 754精度位数 & Kahan灾难性相消

整数误差标准

整数类型要求精确匹配
threshold = 0

针对量化类算子
threshold = 1

Inf/Nan值误差标准

- FP32 → FP16 截断可能产生 Inf (超过 65504)
- 单边 Inf 时, 替换为 dtype 最大有限值进行比较
- 双方都是 Inf 且符号相同 → 视为匹配
- NaN值位置必须一致



评测层-性能基准&对比方案

工具: 使用 Profiler 工具采集真实的NPU内核运行时间。剥离受主机环境影响较大的Host-Python端调度和数据传输的额外开销，精准衡量设备端的真实算力

技术保障: 采用 L2 清缓存、升频预热等底层优化技术，排除环境干扰，确保每次性能测量都在稳定、一致的硬件状态下进行，显著提升数据可重复性

🕒 硬件理论极限 (T_HW)

$$T_{HW} = \min_{\delta} \max_{k \in K} t_k(\delta)$$

$K = \{ \text{Cube, Vector, MTE1, MTE2, MTE3, FixP} \}$ 是六个并发执行单元; $t_k(\delta)$ 是单元 k 在实现配置 δ 下的工作时间。六单元时间维度 overlap。**假设各单元跑到 spec 峰值、完美 overlap、无 launch 开销 — 任何实现不可突破。**

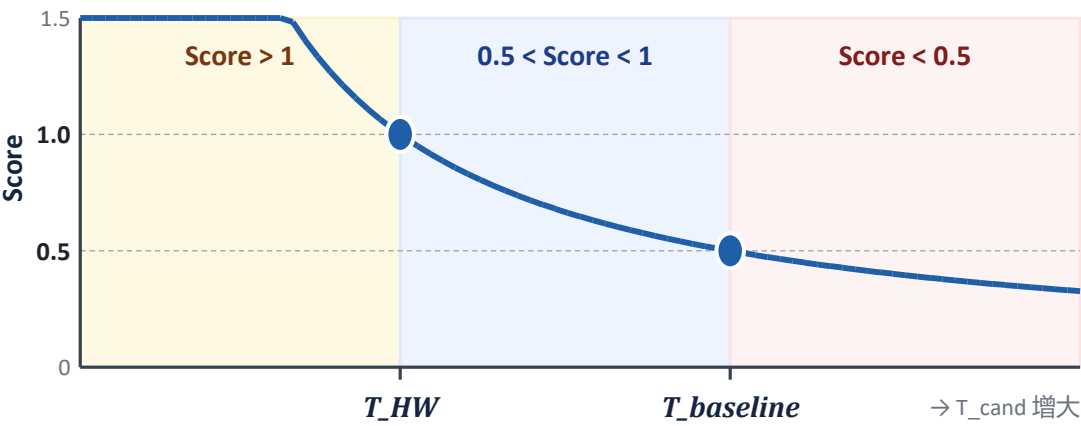
例: $Y = \exp(X)$ 的 T_{HW} 推导

$X, Y \in \text{FP16}^{[1024, 1024]}$ $N = 1.05 \text{ M elements}$ bytes = 2.10 MB(读)+ 2.10 MB(写)

单元 k	工作量 / 峰值	t_k
Cube	无 matmul	0 μs
Vector	1.05 M ops $\div$ 6 TFLOPS (FP16 special)	0.175 μs
MTE1	L1 $\leftrightarrow$ L0 无传输	0 μs
MTE2	2.10 MB $\div$ 1.92 TB/s (GM read)	1.09 μs
MTE3	2.10 MB $\div$ 6.4 TB/s (L2 write)	0.328 μs
FixP	无 cube 写回路径	0 μs

取 max, 即 bottleneck:
 $T_{HW} = \max(0, 0.175, 0, 1.09, 0.328, 0)$ **1.09 μs**

🏗️ 性能对比公式



$$\text{Score}_i = \frac{T_{\text{baseline}} - T_{\text{HW}}}{(T_{\text{cand}} - T_{\text{HW}}) + (T_{\text{baseline}} - T_{\text{HW}})}$$

T_{cand} 是 AI 提交的实测时间。公式将 T_{baseline} 对齐到 0.5, T_{HW} 对齐到 1.0, 衰减到 0。

- Score > 1** 击穿基于原 desc 推导的 T_{HW} 。可能:(a) 测量异常或 reward hacking;(b) AI 发现原 desc 算法之外的等价路径(算法层面创新) — 触发审查,而非自动判错。
- 0.5 < Score < 1** 在原算法范围内, AI 对 baseline 实现做出了实质优化。Score 越接近 1, 越逼近硬件极限。
- Score < 0.5** AI 提交比参考实现还慢。功能正确仍可计 Score, 但表明 AI 实现质量差于 CANN baseline。



评测层-评分体系



逐用例进行评测，综合用例得分汇总成算子级别评分，最终可以聚合成各种维度的评分



用例级维度评分

◆ 编译正确性 ($W_c = 20\%$)

重点：算子代码能否正常编译通过

指标：Pass / Fail (0或1)

◆ 精度正确性 ($W_f = 30\%$)

重点：计算结果是否准确，覆盖测试用例

指标：用例精度通过 (0或1)

◆ 性能优化性 ($W_p = 50\%$)

重点：相比基准实现和硬件极限的加速效果

指标：Score



算子级综合评分

$$\text{Op Score} = \Sigma (W_c \times \text{Compile} + W_f + W_p \times \text{Score}) / \text{Total Cases}$$

1. 编译基础分: $\text{Compile_Pass} (0/1) \times 2$
2. 精度固定分: $\text{通过用例} (0/1) \times 3$
3. 性能浮动分: $\text{通过用例} (\text{Score}) \times 5$
4. 算子分数按总用例数平均



结果聚合规则



Level-N 级别的得分

该级别下包含的所有算子的

综合评分之和 (Sum of Scores)



最终 Benchmark 总分

所有算子综合评分的累加总和，

反映整体评测的最终表现。

评测层-评测工程

CANN-Bench整体架构遵循“分层解耦、模块化插件化扩展”的核心理念，在确保系统稳定性的同时，赋予了极高的灵活性与可维护性

■ 分层设计体系

- 顶层入口 · 交互接口：提供友好的 CLI 入口和配置文件。
- 核心评测 · 逻辑编排&功能实现：负责任务调度与生命周期管理，多层评测、数据加载、报告生成、安全防护。
- 工具支持 · 功能实现：参数构建、路径解析、类型映射等工具。
- 底层硬件 · 硬件交互：对接CANN工具链和硬件交互。

■ 模块化扩展能力

- 精度评测模块：支持快速定义并集成新的测试场景与业务逻辑。
- 性能评测模块：灵活适配不同的性能指标采集工具，数据来源无限制。
- 报告生成模块：支持一键生成 HTML、JSON、Excel 等多种格式的分析报告。

CANN-Bench 系统架构概览



平台层-评测报告

Ascend C 算子生成评测报告

生成模型: Qwen3-72B
评测时间: 2026-05-06 14:30:25
评测用例数: 120 (30个算子 × 4个用例)
评测框架: cann-bench v1.2.0
NPU设备: Ascend 910B

一、评测概览

1.1 评测维度

评测维度	权重	说明
编译正确性	Wc=2	是否编译通过
功能正确性	Wf=3	通过精度用例的数量
性能优化性	Wp=5	验证性能/测试基准性能

评分公式: 单算子综合评分 = 编译通过得分 + 功能通过用例数 × (功能得分 + 性能得分)

1.2 整体表现

指标	数值
编译通过率	90.0% (27/30)
功能通过率	75.8% (91/120)
平均性能加速比	0.45x
综合得分	382.5
总排名	🏆 第2名 (共8个模型)

二、详细评测结果

2.1 Level 1 - 基础算子评测

2.1.1 逐元素激活函数类

算子	编译	精度通过率	平均加速比	综合得分	主要问题
Mish	✅	100% (4/4)	0.62x	15.5	无
Sigmoid	✅	100% (4/4)	0.58x	15.2	无
Gelu	✅	100% (4/4)	0.55x	14.8	无
Exp	✅	100% (4/4)	0.71x	17.1	无

2.1.2 其他基础算子

算子	编译	精度通过率	平均加速比	综合得分	主要问题
ForeachNorm	✅	75% (3/4)	0.35x	8.2	部分场景TensorList处理异常
MaskedScale	✅	100% (4/4)	0.48x	13.4	无
ForeachAddcdiv Scalar	✅	87.5% (7/8)	0.42x	11.6	边界条件处理不完整
SwiGLU	✅	87.5% (7/8)	0.45x	12.3	大shape场景精度损失

三、编译正确性评估

3.1 编译失败统计

算子	编译状态	错误类型	错误详情
UnsortedSegmentSum (L2)	❌ 失败	内存对齐错误	UB buffer alignment error: address must be 32-byte aligned
DynamicQuant (L2)	❌ 失败	数据类型不支持	bfloat16 is not supported in current CANN version
Conv3DBackpropFilter (L3)	❌ 失败	模板实例化错误	template instantiation failed for shape (16, 32, 64, 64, 64)
SparseFlashAttention (L4)	❌ 失败	CANN版本不兼容	requires CANN 8.0.RC1+, current version is 7.0.1
MLA (L4)	❌ 失败	内存布局不兼容	NZ layout is not supported for matmul fusion

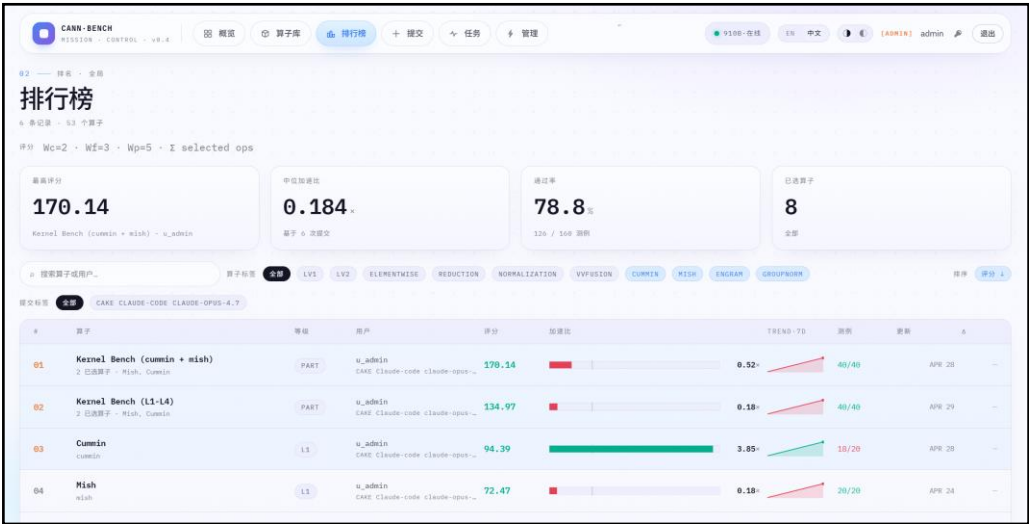
3.2 编译失败根因分析

失败原因	算子数	占比	改进建议
内存对齐问题	1	20%	增强UB内存对齐约束检查
数据类型支持	1	20%	提示词明确数据类型兼容性要求
模板实例化	1	20%	增加动态shape场景处理逻辑
CANN版本依赖	1	20%	明确CANN版本要求, 提供版本检测代码
内存布局	1	20%	增加内存布局转换逻辑

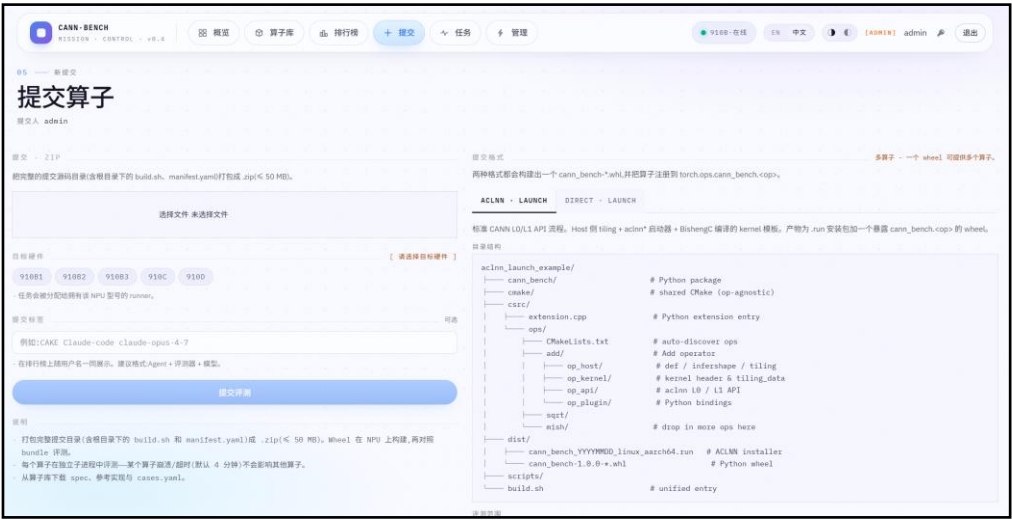
平台层 - 在线评测平台



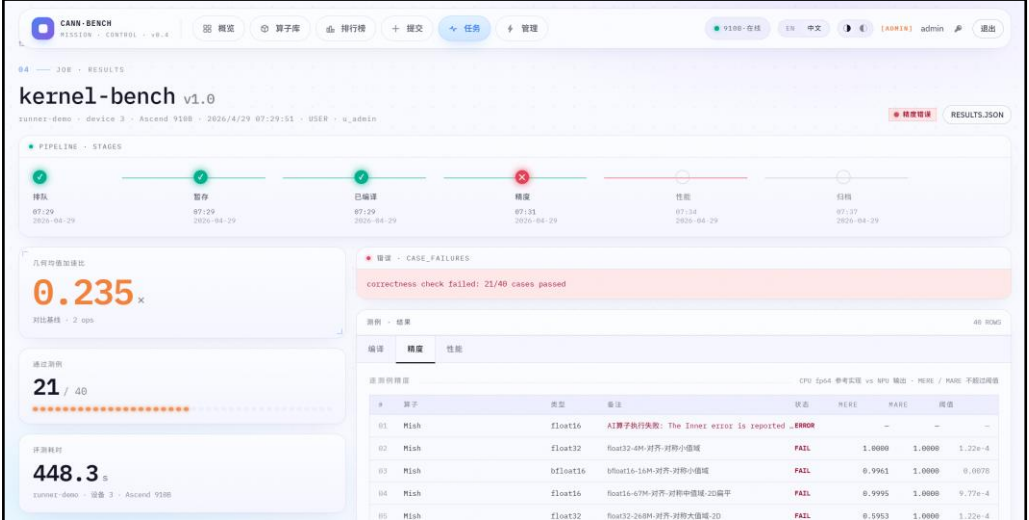
CANN-Bench评测平台 入口界面



CANN-Bench评测平台 排行榜



CANN-Bench评测平台 提交入口



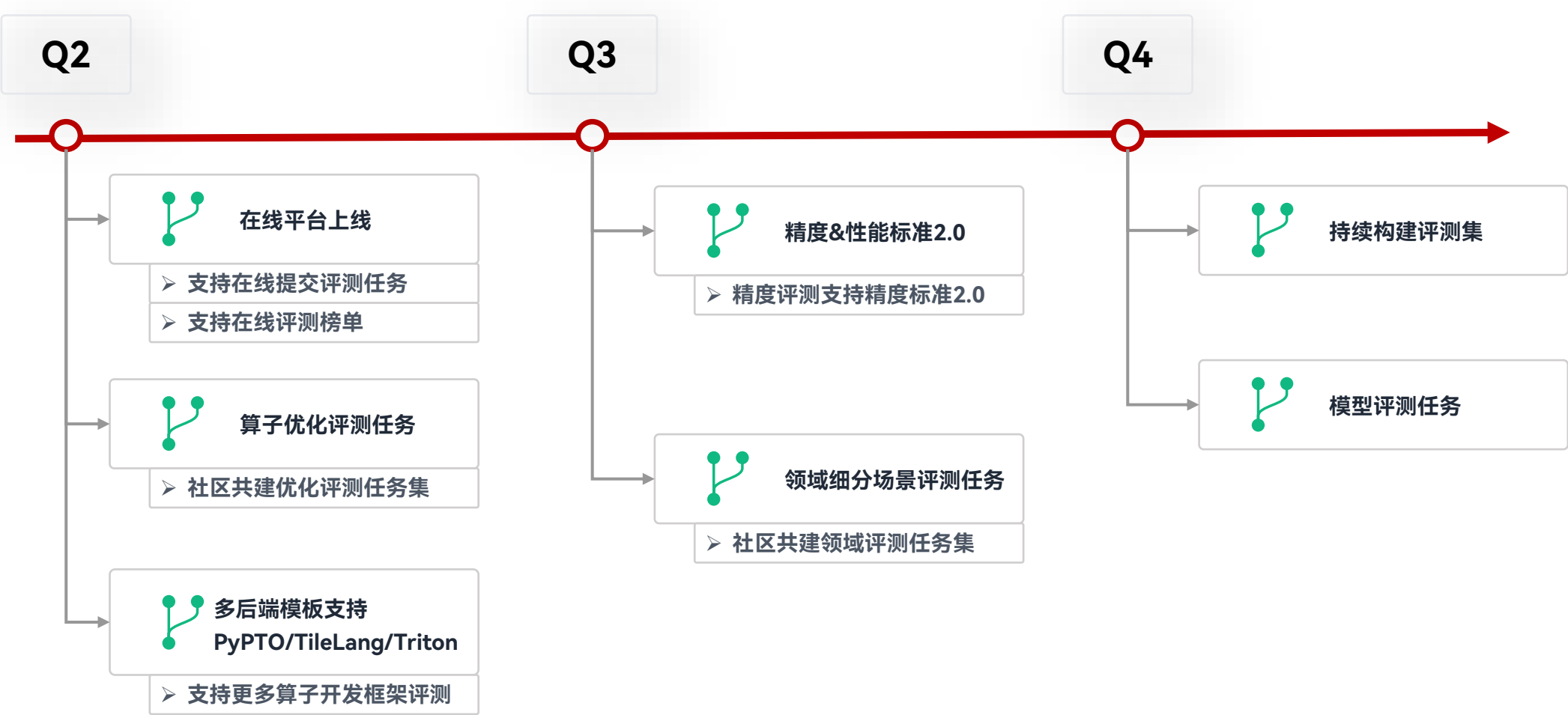
CANN-Bench评测平台 任务进展

<https://gitcode.com/cann/cann-bench>



CANN-Bench 关键里程碑

随着模型能力提升，CANN-Bench每个季度都会更新新的评测任务集



cann-cann-robot

sync(dev): batch port of all pending cann-bench-dev changes

d331e8da

创建于 1 天前

35次提交

文件	最后提交记录	最后更新时间
bench_lab	增加tests目录, 删除冗余目录 Co-authored-by: suyueming<suyuemi...	15 天前
docs	sync(dev): batch port of all pending cann-bench-dev changes Co-au...	1 天前
examples	chore: misc cleanup — .gitignore, extension.cpp license block, moe...	14 天前
kernel_bench	sync(dev): batch port of all pending cann-bench-dev changes Co-au...	1 天前
scripts	sync(dev): batch port of all pending cann-bench-dev changes Co-au...	1 天前
src	sync(dev): batch port of all pending cann-bench-dev changes Co-au...	1 天前
tests	sync(dev): batch port of all pending cann-bench-dev changes Co-au...	1 天前
.gitignore	sync(dev): batch port of all pending cann-bench-dev changes Co-au...	1 天前
LICENSE	add kernel bench specs&golden&cases	30 天前
README.md	sync(dev): batch port of all pending cann-bench-dev changes Co-au...	1 天前
requirements.txt	update kernel list	28 天前

README

Others

cann-bench: CANN 领域评测

评测AI在处理CANN领域代码任务的能力, 涵盖算子生成、算子优化等领域, 支撑模型选型、训练效果评估, 统一量化评估标准, 识别Agent能力短板, 构建CANN领域评测平台, 推动AI能力在CANN领域的持续演进。



CANN-Bench



CANN-Bench微信群



Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上CANN社区获取干货



关注CANN公众号获取资讯